

Лекция 3. Введение в нейронные сети

Дополнительные главы машинного обучения

Автор занятия: Никитин Илья Сергеевич

Образовательная программа
«Машинное обучение в цифровом продукте»

Оглавление

1) Линейная модель, нейрон и многослойная сеть

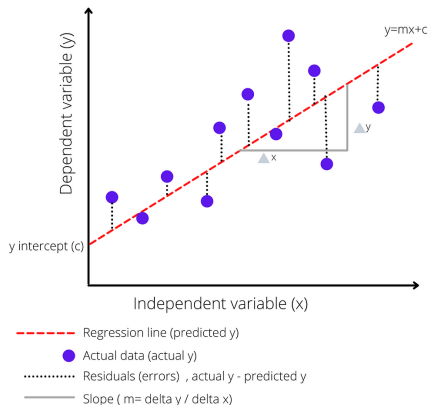
2) Функции потерь

3) Градиентный спуск

4) Обратное распространение ошибки

5) ResNet, BatchNorm и Dropout

Линейная регрессия



Для объекта $x \in \mathbb{R}^d$ предсказываем вещественное значение:

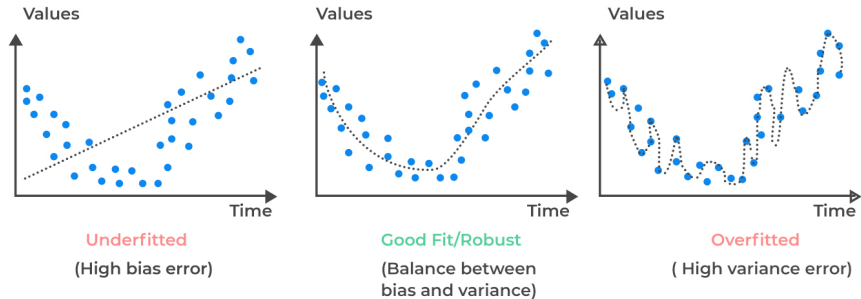
$$\hat{y} = w^\top x + b.$$

Вес w_j задаёт вклад признака, b — свободный член.

При фиксированных остальных признаках прирост прогноза равен $w_j \Delta x_j$.

Подбор коэффициентов меняет наклон и сдвиг, но не форму зависимости.

Когда линейная модель недообучается



Слева прямая слишком проста для зависимости в данных: это **недообучение**. В центре модель описывает общую закономерность. Справа она подстраивается под отдельные точки и шум.

Здесь признак один. Нелинейные признаки можно задать вручную; в нейронной сети будем обучать их вместе с моделью.

Ёмкость модели и качество на новых данных

Хотим, чтобы выбранная сеть могла точно подогнать обучающую выборку, вплоть до переобучения. Иначе её выразительности может не хватить даже для train.

- Если ошибка на train остаётся большой, выясняем причину: ограничение модели или трудности обучения.
- Насколько сильно подгонять веса, решаем по качеству на validation.

Обсудим. Ошибка на train почти нулевая. Значит ли это, что модель хорошая?

Ёмкость модели и качество на новых данных

Хотим, чтобы выбранная сеть могла точно подогнать обучающую выборку, вплоть до переобучения. Иначе её выразительности может не хватить даже для train.

- Если ошибка на train остаётся большой, выясняем причину: ограничение модели или трудности обучения.
- Насколько сильно подгонять веса, решаем по качеству на validation.

Обсудим. Ошибка на train почти нулевая. Значит ли это, что модель хорошая?

Ответ. Это ещё нужно проверить на новых данных. Модель могла выучить закономерность, а могла запомнить обучающие примеры.

Искусственный нейрон и перцептрон

$$z = w^T x + b, \quad a = \phi(z).$$

Нейрон складывает взвешенные входы и применяет функцию активации ϕ .

- Порог $\phi(z) = \mathbf{1}[z \geq 0]$ даёт классический перцептрон.
- Сигмоида $\sigma(z) = 1/(1 + e^{-z})$ даёт гладкий выход в $(0, 1)$.

Производная порога почти везде нулевая. Для градиентного обучения нужны другие активации.

Обсудим. Что изменится, если к выходу линейной регрессии применить сигмоиду?

Искусственный нейрон и перцептрон

$$z = w^{\top} x + b, \quad a = \phi(z).$$

Нейрон складывает взвешенные входы и применяет функцию активации ϕ .

- Порог $\phi(z) = \mathbf{1}[z \geq 0]$ даёт классический перцептрон.
- Сигмоида $\sigma(z) = 1/(1 + e^{-z})$ даёт гладкий выход в $(0, 1)$.

Производная порога почти везде нулевая. Для градиентного обучения нужны другие активации.

Обсудим. Что изменится, если к выходу линейной регрессии применить сигмоиду?

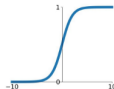
Ответ. Получим нелинейную функцию $\sigma(w^{\top} x + b)$ с выходом в $(0, 1)$. Её можно интерпретировать как вероятность положительного класса.

Активации и их производные

Activation Functions

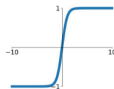
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



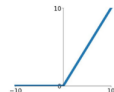
tanh

$$\tanh(x)$$



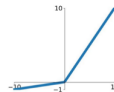
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

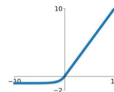


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



$$\sigma'(z) = \sigma(z)(1 - \sigma(z)), \quad \tanh'(z) = 1 - \tanh^2(z), \quad \text{ReLU}'(z) = \mathbf{1}[z > 0].$$

Сигмоида и tanh насыщаются. У ReLU производной в нуле нет; в backprop обычно выбирают ноль.

Зачем активация между слоями

Два последовательных аффинных преобразования:

$$h = W_1 x + b_1, \quad \hat{y} = W_2 h + b_2.$$

После подстановки:

$$\hat{y} = \underbrace{W_2 W_1}_W x + \underbrace{W_2 b_1 + b_2}_b.$$

Без нелинейности сеть любой глубины остаётся аффинной функцией.

Обсудим. Сможет ли такая сеть описать параболу $y = x^2$, если добавить ещё линейных слоёв?

Зачем активация между слоями

Два последовательных аффинных преобразования:

$$h = W_1 x + b_1, \quad \hat{y} = W_2 h + b_2.$$

После подстановки:

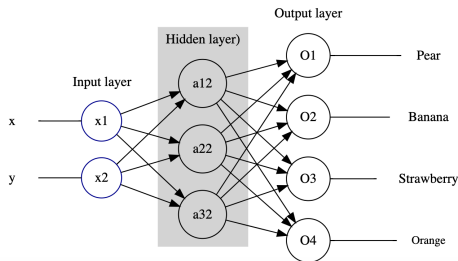
$$\hat{y} = \underbrace{W_2 W_1}_W x + \underbrace{W_2 b_1 + b_2}_b.$$

Без нелинейности сеть любой глубины остаётся аффинной функцией.

Обсудим. Сможет ли такая сеть описать параболу $y = x^2$, если добавить ещё линейных слоёв?

Ответ. Нет. Композиция останется прямой $wx + b$. Чтобы наклон менялся вместе с x , нужна нелинейная активация между слоями.

Многослойный перцептрон



$W_l \in \mathbb{R}^{d_l \times d_{l-1}}$ — матрица весов.

$b_l, z^{(l)}, h^{(l)} \in \mathbb{R}^{d_l}$ — векторы-столбцы.

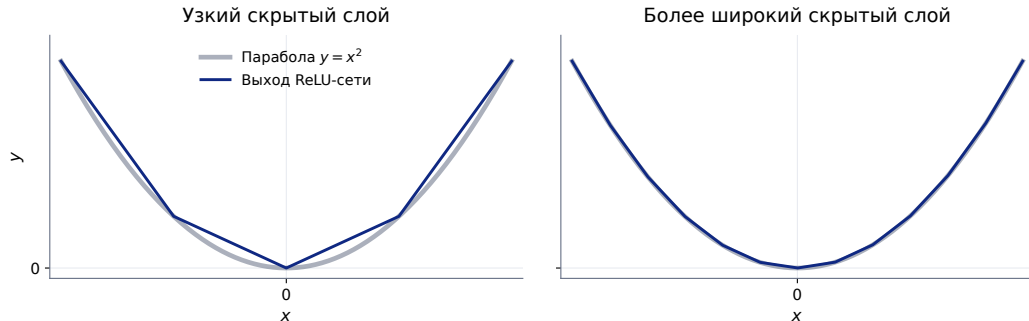
$$h^{(0)} = x,$$

$$z^{(l)} = W_l h^{(l-1)} + b_l,$$

$$h^{(l)} = \phi_l(z^{(l)}).$$

Каждый слой применяет аффинное преобразование, затем активацию. Скрытые слои строят признаки, выходной слой формирует прогноз. Все веса обучаются совместно.

Аппроксимация параболы

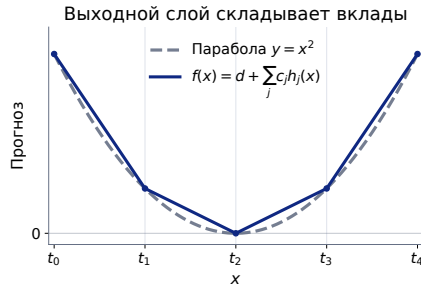
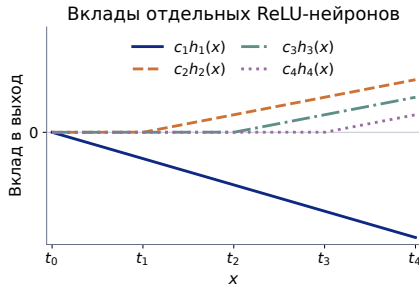


ReLU-сеть строит кусочно-линейную функцию. Больше скрытых нейронов позволяет точнее приблизить кривую на заданном отрезке.

Здесь веса выбраны для иллюстрации. При обучении их подбирает оптимизатор по ошибке на данных.

Парабола как сумма ReLU-нейронов

Скрытый слой: $h_j(x) = \text{ReLU}(a_jx + b_j)$. Выход: $f(x) = d + \sum_j c_j h_j(x)$.



Между узлами сумма линейна: $f(x) = s_kx + q_k$. Включение следующего нейрона меняет её наклон.

В узлах функция совпадает с параболой, а между ними приближает её.

Оглавление

1) Линейная модель, нейрон и многослойная сеть

2) Функции потерь

3) Градиентный спуск

4) Обратное распространение ошибки

5) ResNet, BatchNorm и Dropout

Что минимизирует нейронная сеть

Есть обучающая выборка $\{(x_i, y_i)\}_{i=1}^n$ и параметры θ :

$$\hat{y}_i = f_{\theta}(x_i), \quad J(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(\hat{y}_i, y_i) + \lambda \Omega(\theta).$$

- ℓ (Loss) измеряет ошибку на одном объекте.
- Среднее измеряет ошибку на выборке.
- Ω — регуляризационный член, $\lambda \geq 0$ задаёт силу регуляризации.

Для вывода `backprop` сначала положим $\lambda = 0$.

Регрессия: квадратичная потеря

Для скалярного неограниченного прогноза выход часто оставляют линейным:

$$\ell(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2, \quad \frac{\partial \ell}{\partial \hat{y}} = \hat{y} - y.$$

Если $y \mid x \sim \mathcal{N}(f_\theta(x), \sigma^2)$ с фиксированной дисперсией, максимизация правдоподобия приводит к минимизации суммы квадратов.

- Крупные ошибки получают большой вес, поэтому выбросы влияют сильнее.
- Коэффициент $1/2$ упрощает производную. Без регуляризации положение минимума не меняется.

Бинарная классификация: cross-entropy

Логит $s \in \mathbb{R}$, вероятность $p = \sigma(s)$, метка $y \in \{0, 1\}$:

$$P(y \mid x) = p^y(1 - p)^{1-y}.$$

Отрицательный логарифм правдоподобия даёт BCE:

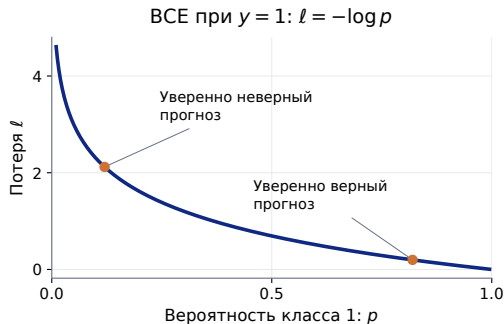
$$\ell = -y \log p - (1 - y) \log(1 - p).$$

Для вычислений устойчивее эквивалентная запись по логиту:

$$\ell = \max(s, 0) - ys + \log(1 + e^{-|s|}).$$

Так не приходится отдельно вычислять $\log(0)$ при округлении вероятности.

Как Loss штрафует ошибку



Квадратичная потеря быстро растёт с величиной ошибки. Cross-entropy особенно сильно штрафует уверенный прогноз в пользу неверного класса.

Несколько классов: softmax и cross-entropy

Для K взаимоисключающих классов сеть выдаёт логиты $s_1, \dots, s_K$:

$$p_k = \frac{e^{s_k}}{\sum_j e^{s_j}}, \quad \ell = - \sum_k y_k \log p_k = - \log p_c.$$

y — one-hot-вектор, c — правильный класс. Для устойчивости:

$$p_k = \frac{e^{s_k - m}}{\sum_j e^{s_j - m}}, \quad m = \max_j s_j.$$

Сдвиг всех логитов не меняет вероятности.

Если метки могут встречаться одновременно, обычно нужны отдельные сигмоиды и BCE по каждой метке.

Выход модели, Loss и метрика

Задача	Выход для прогноза	Loss
Регрессия	Число	MSE
Два класса	Сигмоида	BCE
Один из K классов	Softmax	Cross-entropy
Несколько меток	K сигмоид	Сумма BCE

Обсудим. Почему обычно не обучают сеть напрямую по accuracy?

Выход модели, Loss и метрика

Задача	Выход для прогноза	Loss
Регрессия	Число	MSE
Два класса	Сигмоида	BCE
Один из K классов	Softmax	Cross-entropy
Несколько меток	K сигмоид	Сумма BCE

Обсудим. Почему обычно не обучают сеть напрямую по accuracy?

Ответ. Пороговый ответ не меняется при малых изменениях весов: производная почти везде нулевая. Loss даёт сигнал о том, как изменить уверенность прогноза.

Оглавление

1) Линейная модель, нейрон и многослойная сеть

2) Функции потерь

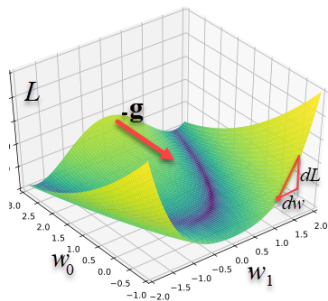
3) Градиентный спуск

4) Обратное распространение ошибки

5) ResNet, BatchNorm и Dropout

Градиент и шаг обучения

$$\theta_{t+1} = \theta_t - \eta_t \nabla_{\theta} J(\theta_t), \quad \eta_t > 0.$$



Градиент содержит производную по каждому параметру.

- Направление $-\nabla J$ локально уменьшает Loss.
- Learning rate η задаёт размер шага.
- Слишком большой шаг может увеличить Loss.

Поверхность Loss у нейронной сети обычно невыпуклая. Глобальный минимум не гарантирован.

Размер шага: сходимость и расходимость



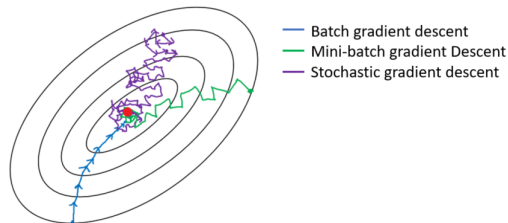
Слева шаг позволяет постепенно приблизиться к минимуму. В центре возникают затухающие колебания. Справа слишком большой шаг уводит всё дальше.

Допустимый learning rate зависит от кривизны Loss и может меняться по ходу обучения.

Mini-batch SGD

Вместо всей выборки берём подмножество $\mathcal{B}_t$ размера B :

$$g_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla_{\theta} \ell_i, \quad \theta_{t+1} = \theta_t - \eta_t g_t.$$



Полный GD использует все объекты. Мини-батч даёт более дешёвую, но шумную оценку градиента.

Эпоха — один проход по обучающей выборке.

В среднем по независимо и равномерно выбранным батчам оценка даёт полный градиент среднего Loss без регуляризации. Отдельный шаг может увеличить полный Loss.

Momentum

Momentum усредняет градиенты последних шагов:

$$m_t = \beta m_{t-1} + (1 - \beta)g_t, \quad m_0 = 0, \quad 0 \leq \beta < 1,$$

$$\theta_{t+1} = \theta_t - \eta m_t.$$

- Если градиенты согласованы, направление движения сохраняется.
- Колебания градиента из стороны в сторону частично усредняются.
- Чем ближе β к единице, тем дольше учитывается история.

Встречается запись без множителя $(1 - \beta)$: тогда иначе масштабируется эффективный learning rate.

Adam: адаптивный шаг по координатам

Для градиента g_t в точке θ_{t-1} храним два скользящих средних:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t \odot g_t.$$

Начинаем с $m_0 = v_0 = 0$. Корректируем смещение начальных оценок:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad \theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

- m_t усредняет направление, v_t оценивает масштаб квадратов градиентов.
- Деление и корень выполняются по координатам; $\epsilon > 0$ стабилизирует деление.
- $\beta_1, \beta_2 \in [0, 1)$ задают память, общий η по-прежнему нужно выбирать.

Kingma, Ba, Adam: A Method for Stochastic Optimization, 2014/2015.

Обучение и проверка на новых объектах

- На **train** вычисляем градиенты и подбираем веса.
- На **validation** выбираем архитектуру, learning rate, регуляризацию и момент остановки.
- **Test** используем для итоговой оценки после выбора модели.

Статистики масштабирования признаков оцениваем только на train.

Обсудим. Train Loss падает, а validation Loss растёт. Что происходит?

Обучение и проверка на новых объектах

- На **train** вычисляем градиенты и подбираем веса.
- На **validation** выбираем архитектуру, learning rate, регуляризацию и момент остановки.
- **Test** используем для итоговой оценки после выбора модели.

Статистики масштабирования признаков оцениваем только на train.

Обсудим. Train Loss падает, а validation Loss растёт. Что происходит?

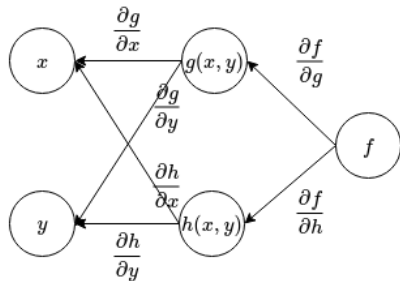
Ответ. Возможное переобучение. Полезны ранняя остановка, регуляризация и проверка данных. Продолжение оптимизации train само по себе проблему скорее всего не решит.

Оглавление

- 1) Линейная модель, нейрон и многослойная сеть
- 2) Функции потерь
- 3) Градиентный спуск
- 4) Обратное распространение ошибки**
- 5) ResNet, BatchNorm и Dropout

Вычислительный граф

Сложную функцию разбиваем на операции. Здесь сначала вычисляем $g(x, y)$ и $h(x, y)$, затем $f = F(g, h)$.

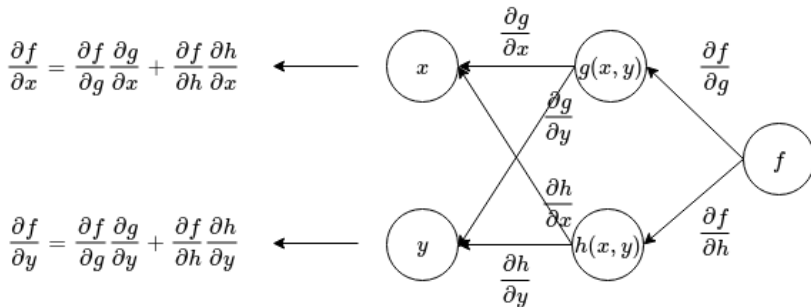


Прямой проход: от x, y к f . Сохраняем промежуточные значения.

Обратный проход: от f к x, y . Передаём производные через каждую операцию. Стрелки на рисунке показывают именно обратный проход.

У нейросети такими операциями служат слои, активации и Loss. Обратный проход начинается с $\partial \ell / \partial \ell = 1$.

Правило цепочки и несколько путей



- Вдоль одного пути локальные производные перемножаются.
- Если путей несколько, их вклады складываются.

Backprop применяет это правило в каждом узле и переиспользует уже вычисленные производные.

Обратный проход через линейный слой

Пусть $z = Wh + b$, известен входящий градиент $\delta = \partial\ell/\partial z$.

$$\boxed{\frac{\partial\ell}{\partial W} = \delta h^\top, \quad \frac{\partial\ell}{\partial b} = \delta, \quad \frac{\partial\ell}{\partial h} = W^\top \delta.}$$

- Для веса W_{jk} : $\partial\ell/\partial W_{jk} = \delta_j h_k$.
- Для входа h_k : суммируем $W_{jk}\delta_j$ по всем выходным нейронам.

Если $W \in \mathbb{R}^{m \times d}$, то $\delta h^\top \in \mathbb{R}^{m \times d}$, а $W^\top \delta \in \mathbb{R}^d$. Размерности совпадают с параметрами и входом.

Рекурсия по скрытым слоям

Обозначим $\delta^{(l)} = \partial \ell / \partial z^{(l)}$. Тогда

$$\delta^{(l)} = (W_{l+1}^\top \delta^{(l+1)}) \odot \phi'_l(z^{(l)}).$$

$\odot$ — покоординатное умножение. После этого:

$$\frac{\partial \ell}{\partial W_l} = \delta^{(l)} (h^{(l-1)})^\top, \quad \frac{\partial \ell}{\partial b_l} = \delta^{(l)}.$$

У ReLU маска $\mathbf{1}[z^{(l)} > 0]$ пропускает градиент через активные нейроны. У сигмиды множитель $\sigma(z)(1 - \sigma(z)) \leq 1/4$ может сильно уменьшать его.

Прямой проход в PyTorch

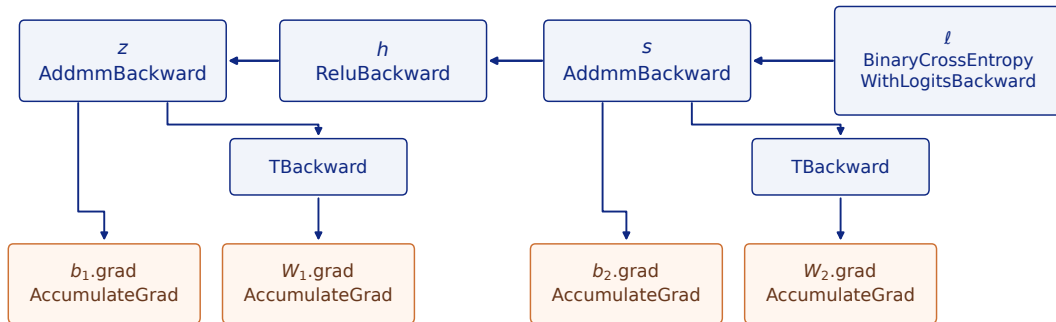
```
model = nn.Sequential(  
    nn.Linear(2, 3), nn.ReLU(), nn.Linear(3, 1)  
)  
criterion = nn.BCEWithLogitsLoss()  
logits = model(X)  
loss = criterion(logits, y)
```

В X объекты записаны строками: форма $(B, 2)$. У y форма $(B, 1)$; бинарные метки записаны числами с плавающей точкой.

- У параметров сети `requires_grad=True`.
- Результат операции хранит ссылку `grad_fn` на её обратный проход.
- Связи `next_functions` ведут к предыдущим операциям.

Так при выполнении кода возникает граф для вычисления производных.

Граф autograd



Граф для кода на предыдущем слайде. Стрелки направлены по обратному проходу: от Loss к параметрам. Узлы AccumulateGrad добавляют вклад в `parameter.grad`.

Градиенты и обновление весов

```
optimizer.zero_grad(set_to_none=True)
loss = criterion(model(X), y)
loss.backward()
optimizer.step()
```

- `backward()` начинает с $\partial \ell / \partial \ell = 1$, применяет локальные производные и складывает вклады разных ветвей.
- Градиенты накапливаются в `.grad`; веса меняет `step()`.
- Новый прямой проход строит новый граф. Сохранённые промежуточные значения обычно освобождаются после `backward()`.

При `X.requires_grad=False` градиент по X не нужен, но сами значения X нужны для производных по весам.

Оглавление

- 1) Линейная модель, нейрон и многослойная сеть
- 2) Функции потерь
- 3) Градиентный спуск
- 4) Обратное распространение ошибки
- 5) ResNet, BatchNorm и Dropout**

Глубина и масштаб градиента

При движении к ранним слоям градиент многократно умножается на матрицы весов и производные активаций:

$$\nabla_{h^{(0)}} \ell = J_1^\top J_2^\top \cdots J_L^\top \nabla_{h^{(L)}} \ell.$$

- Если преобразования сжимают сигнал, ранние слои получают всё меньший градиент.
- Если они слишком сильно его растягивают, градиент может взрываться.
- Насыщение активаций также уменьшает локальные производные.

Масштаб начальных весов определяет, с какими активациями и градиентами начнётся обучение.

Почему начальные веса должны различаться

У двух одинаково инициализированных скрытых нейронов одинаковые выходы. Если совпадают и их связи со следующим слоем, одинаковыми будут и градиенты.

$$w_1 = w_2, \quad b_1 = b_2 \quad \implies \quad h_1(x) = h_2(x).$$

После обновления нейроны остаются одинаковыми и продолжают учить один и тот же признак.

- Независимые случайные веса нарушают эту симметрию.
- Смещения при этом часто можно начать с нуля.
- Случайность нужна вместе с правильно выбранным масштабом весов.

Инициализация Xavier и He

Для $z_j = \sum_{i=1}^{d_{\text{in}}} W_{ji} h_i$ при независимых центрированных весах:

$$\text{Var}(z_j) \approx d_{\text{in}} \text{Var}(W_{ji}) \mathbb{E}[h_i^2].$$

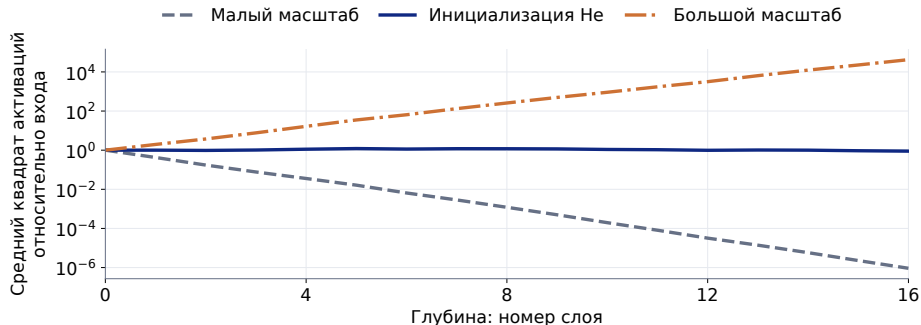
Чтобы масштаб сигнала не менялся резко от слоя к слою, учитываем число входов и активацию.

- **Xavier:** $\text{Var}(W_{ji}) = 2/(d_{\text{in}} + d_{\text{out}})$, компромисс для прямого и обратного проходов при близкой к линейной активации.
- **He:** $\text{Var}(W_{ji}) = 2/d_{\text{in}}$ для ReLU. Множитель 2 компенсирует обнуление примерно половины симметричного сигнала.

Это начальный выбор масштаба, основанный на приближённых предпосылках.

Glorot, Bengio, Understanding the Difficulty of Training Deep Feedforward Neural Networks, 2010; He et al., Delving Deep into Rectifiers, 2015.

Как начальный масштаб меняется по глубине



Слишком малые веса гасят активации, слишком большие усиливают их от слоя к слою. Инициализация He учитывает ReLU и помогает сохранить масштаб.

Здесь показан прямой проход случайных ReLU-сетей до обучения, без смещений. Кривые усреднены по нескольким инициализациям.

ResNet: остаточная связь

В остаточном блоке к выходу обучаемой ветви прибавляют вход:

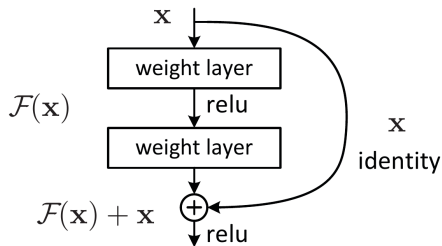
$$u = h + F(h).$$

Блок учит **поправку** к исходному представлению.

Если представление уже подходит, ветвь может приблизить $F(h) \approx 0$ и сохранить его в сумме.

На рисунке после суммы стоит ReLU:

$h_{\text{next}} = \text{ReLU}(u)$. Складываемые тензоры имеют одинаковую форму.



He et al., Deep Residual Learning for Image Recognition, рис. 2.

Почему остаточная связь помогает обучению

Для суммы $u = h + F(h)$ градиент имеет два слагаемых:

$$\nabla_h \ell = \underbrace{\nabla_u \ell}_{\text{прямой путь}} + \underbrace{J_F(h)^\top \nabla_u \ell}_{\text{через обучаемую ветвь}} .$$

- Часть градиента проходит к входу без умножения на производные внутри F .
- Глубокой сети проще сохранить полезное представление и постепенно уточнять его.
- Дополнительные слои могут учить небольшую поправку вместо полного преобразования заново.

Формула записана для суммы u , до следующей активации. Residual-связи облегчают оптимизацию глубоких сетей.

BatchNorm: нормализация внутри сети

Для одного признака $z_1, \dots, z_B$ в обучающем мини-батче:

$$\mu_B = \frac{1}{B} \sum_i z_i, \quad \sigma_B^2 = \frac{1}{B} \sum_i (z_i - \mu_B)^2,$$

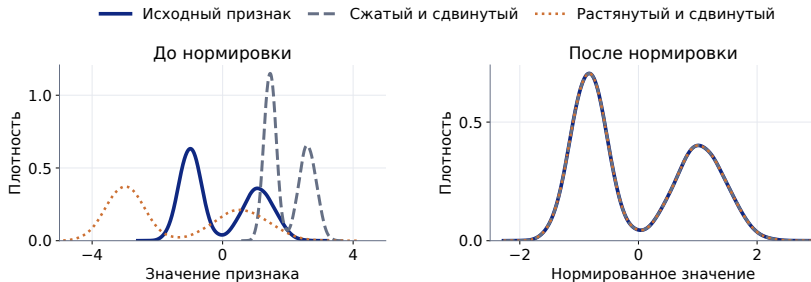
$$\hat{z}_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad \text{BN}(z_i) = \gamma \hat{z}_i + \beta.$$

γ, β обучаются, $\epsilon > 0$ стабилизирует деление.

Статистики считаются **отдельно для каждого признака по батчу**. При обучении выход одного объекта зависит и от других объектов батча.

Ioffe, Szegedy, Batch Normalization, 2015.

Что меняет нормировка признака



Синяя линия — исходный признак. Серая и оранжевая — его сжатая и растянутая копии со сдвигом: $z' = az + b$, $a > 0$.

После вычитания среднего и деления на стандартное отклонение эти копии совпадают. Форма распределения сохраняется. На рисунке показана стандартизация без ϵ и обучаемых γ, β .

BatchNorm: длина и направление весов

Сдвиг b убирается при центрировании. Разделим веса на длину r и направление u :

$$w = ru, \quad r = \|w\| > 0, \quad \|u\| = 1, \quad q_i = u^\top x_i.$$

Тогда $z_i = rq_i$, $\mu_z = r\mu_q$ и $\sigma_z^2 = r^2\sigma_q^2$. После нормировки:

$$\hat{z}_i = \frac{rq_i - r\mu_q}{\sqrt{r^2\sigma_q^2 + \epsilon}} = \frac{q_i - \mu_q}{\sqrt{\sigma_q^2 + \epsilon/r^2}} \approx \frac{q_i - \mu_q}{\sigma_q}.$$

Направление u определяет, какой признак выделяет нейрон. **Длина** r почти сокращается: она одинаково растягивает значения и их стандартное отклонение.

Рассматриваем train-режим, ненулевую дисперсию и малое ϵ относительно дисперсии z . BN нормирует активации, а не сами веса.

Ioffe, Szegedy, Batch Normalization, раздел 3.3, 2015.

Decoupling: признак и его масштаб

После стандартизации применяется обучаемое преобразование:

$$a_i = \gamma \hat{z}_i + \beta.$$

- Поворот вектора w меняет направление u и сам выделяемый признак.
- Изменение длины w почти не меняет нормированный признак.
- Параметр γ регулирует масштаб выхода, β — его средний уровень.

Так обучение направления и настройка амплитуды получают разные параметры. Это уменьшает чувствительность выхода к масштабу предыдущих весов и часто облегчает оптимизацию.

Длина w всё ещё влияет на шаг оптимизатора. Разделение ролей параметров не означает, что они обучаются независимо.

Ioffe, Szegedy, 2015; Arora et al., Theoretical Analysis of Auto Rate-Tuning by Batch Normalization, 2019.

BatchNorm при обучении и применении

Обучение

- Статистики текущего батча.
- Обновление скользящих оценок.
- Градиент проходит и через среднее, и через дисперсию.

При малых батчах оценки могут быть шумными. Перед validation и test в PyTorch вызывают `model.eval()`.

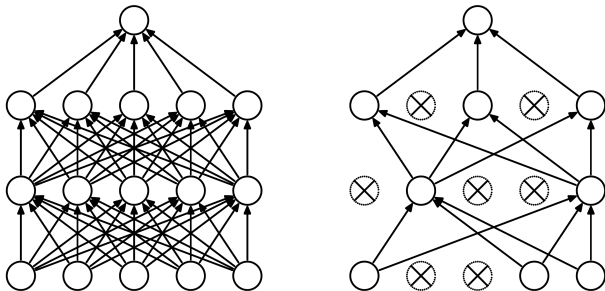
Santurkar et al., How Does Batch Normalization Help Optimization?, 2018.

Применение

- Накопленные статистики train.
- γ, β фиксированы.
- Предсказание не зависит от состава тестового батча.

Dropout

Dropout — способ регуляризации: во время обучения часть активаций случайно заменяют нулями. На каждом прямом проходе выбирается новая маска.



Слева полная сеть, справа один из вариантов после выключения нейронов. Оставшиеся связи используют те же обучаемые веса.

Srivastava et al., Dropout, JMLR, 2014, рис. 1.

Dropout при обучении и применении

Пусть p_{drop} — вероятность выключения, $q = 1 - p_{\text{drop}} > 0$.

Обучение: inverted dropout

$$m_j \sim \text{Bernoulli}(q), \quad \tilde{h}_j = \frac{m_j}{q} h_j.$$

Выключенная активация становится нулём. Оставшиеся увеличиваются, чтобы сохранить средний масштаб:

$$\mathbb{E}[\tilde{h}_j \mid h_j] = h_j.$$

Validation и test

$$\tilde{h}_j = h_j.$$

Случайная маска отключена.
Дополнительное масштабирование не требуется: оно уже учтено при обучении.

В PyTorch этот режим включает `model.eval()`.

Почему Dropout уменьшает переобучение

Нейрон не может постоянно рассчитывать на один и тот же набор соседних признаков: часть из них на текущем проходе отсутствует.

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \mathbb{E}_m [\ell(f_\theta(x_i; m), y_i)].$$

- Оптимизируем качество при разных масках m , используя общие веса θ .
- Это затрудняет запоминание примеров через хрупкие сочетания отдельных признаков.
- Интуитивно обучается множество подсетей, которые разделяют параметры.

Полная сеть при применении лишь приближает усреднение подсетей. Из-за нелинейностей равенство их выходов в общем случае не выполняется.

Какую проблему решает каждый приём

Приём	Основной механизм	Что проверить
ResNet	Сложение входа и поправки, дополнительный путь градиента	Совпадение форм, активация после суммы
BatchNorm	Нормализация по батчу, обучаемый масштаб	Размер батча, статистики и режим применения
Dropout	Случайные маски в обучении, регуляризация	Вероятность выключения, отключение на validation/test

Приёмы добавляют по одному и сравнивают качество на одной и той же validation-выборке.

Литература

- Goodfellow, Bengio, Courville. Deep Learning, главы 6–8: прямые сети, backprop, регуляризация и оптимизация.
- He et al. Deep Residual Learning for Image Recognition, 2015/2016.
- Ioffe, Szegedy. Batch Normalization, 2015.
- Srivastava et al. Dropout: A Simple Way to Prevent Neural Networks from Overfitting, 2014.
- Santurkar et al. How Does Batch Normalization Help Optimization?, 2018.
- PyTorch. Automatic Differentiation with torch.autograd.

Спасибо за внимание!